Wstęp do SQL

Baza danych: PostgreSQL

Przemysław Starosta - 2026

Proste zapytania

Ten kod zwraca wszystkie wiersze (nie określono klauzuli WHERE) i wszystkie kolumny (przy użyciu *)

	123 employee_id	A-Z first_name	A-Z last_name	A-Z email	A-Z phone_number	hire_date	A-Z job_id	123 salary	123 commission_pct	123 manager_id	123 department_id
1	100	Steven	King	SKING	1.515.555.0100	2013-06-17	AD_PRES	24,000	[NULL]	[NULL]	90
2	101	Neena	Yang	NYANG	1.515.555.0101	2015-09-21	AD_VP	17,000	[NULL]	100	90
3	102	Lex	Garcia	LGARCIA	1.515.555.0102	2011-01-13	AD_VP	17,000	[NULL]	100	90
4	103	Alexander	James	AJAMES	1.590.555.0103	2016-01-03	IT_PROG	9,000	[NULL]	102	60
5	104	Bruce	Miller	BMILLER	1.590.555.0104	2017-05-21	IT_PROG	6,000	[NULL]	103	60
6	105	David	Williams	DWILLIAMS	1.590.555.0105	2015-06-25	IT_PROG	4,800	[NULL]	103	60
7	106	Valli	Jackson	VJACKSON	1.590.555.0106	2016-02-05	IT_PROG	4,800	[NULL]	103	60

```
select
*
from employees

--alternatywnie
select
e.*
from employees e
```

SELECT - LIMIT

Wskazuje, że tylko określony pierwszy zestaw zostanie zwrócony z zestawu wyników zapytania

	123 employee_id	A-Z first_name	A-Z last_name	A-Z email
1	100	Steven	King	SKING
2	101	Neena	Yang	NYANG
3	102	Lex	Garcia	LGARCIA
4	103	Alexander	James	AJAMES
5	104	Bruce	Miller	BMILLER
6	105	David	Williams	DWILLIAMS
7	106	Valli	Jackson	VJACKSON
8	107	Diana	Nguyen	DNGUYEN
9	108	Nancy	Gruenberg	NGRUENBE
10	109	Daniel	Faviet	DFAVIET

```
select
*
from Employees
limit 10
```

Wybieranie kolumn

Zwracam wszystkie wiersze (nie określono klauzuli WHERE) i tylko kolumny (employee_id, first_name, last_name, salary, department_id) z tabeli Employees.

	123 employee_id	A-Z first_name	A-Z last_name	123 salary	123 department_id
1	100	Steven	King	24,000	90
2	101	Neena	Yang	17,000	90
3	102	Lex	Garcia	17,000	90
4	103	Alexander	James	9,000	60
5	104	Bruce	Miller	6,000	60
6	105	David	Williams	4,800	60
7	106	Valli	Jackson	4,800	60

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
```

Aliasy

Aliasy kolumn w zapytaniach SQL służą do **tymczasowego nadawania bardziej opisowych lub krótszych nazw kolumnom w zestawie wynikowym zapytania**. Nie zmieniają one nazw kolumn w samej tabeli w bazie danych, a jedynie w wynikach, które widzisz po wykonaniu zapytania

	123 employee_id	A-Z fname	A-Z this is last_name	123 after_add
1	100	Steven	King	24,099
2	101	Neena	Yang	17,099
3	102	Lex	Garcia	17,099
4	103	Alexander	James	9,099
5	104	Bruce	Miller	6,099
6	105	David	Williams	4,899
7	106	Valli	Jackson	4,899

```
select
    employee_id,
    first_name fname,
    last_name as "this is last_name",
    salary + 99 after_add
from employees
```

WHERE

Zwracam tylko wiersze dla first_name, last_name i salary, które mają salary większe od 15000.

	A-Z first_name	A-Z last_name	123 salary
1	Steven	King	24,000
2	Neena	Yang	17,000
3	Lex	Garcia	17,000

```
select
    first_name,
    last_name,
    salary
from Employees
where salary > 15000
```

LIKE(1/3)

Funkcja **LIKE** w SQL służy do wyszukiwania wierszy w tabeli, które pasują do określonego wzorca tekstowego.

Zamiast szukać dokładnego dopasowania (jak przy użyciu operatora =), LIKE pozwala na użycie symboli wieloznacznych (wildcard characters), które reprezentują jeden lub wiele znaków.

Dwa główne symbole wieloznaczne używane z LIKE to:

- **% (procent): Reprezentuje zero lub więcej dowolnych znaków.**
  - WHERE nazwa_kolumny LIKE 'a%': Znajdzie wszystkie wartości w kolumnie nazwa_kolumny, które zaczynają się na literę "a" (np. "adam", "ania", "arbuz", "a").
  - WHERE nazwa_kolumny LIKE '%tekst%': Znajdzie wszystkie wartości, które zawierają w sobie ciąg znaków "tekst" (np. "jakiś tekst", "tekst na początku", "koniec tekstu").
  - WHERE nazwa_kolumny LIKE 'p%o': Znajdzie wszystkie wartości, które zaczynają się na "p" i kończą na "o" (np. "piwo", "piano", "polo").

- **(podkreślenie): Reprezentuje dokładnie jeden dowolny znak.**
  - WHERE nazwa_kolumny LIKE 'k_t': Znajdzie wszystkie wartości, które zaczynają się na "k", mają jeden dowolny znak w środku i kończą się na "t" (np. "kot", "kit", "kat").
  - WHERE nazwa_kolumny LIKE '_a%': Znajdzie wszystkie wartości, które mają dowolny pierwszy znak, po którym następuje "a" i cokolwiek dalej (np. "bartek", "california").

LIKE(2/3)

	employee_id	first_name	last_name	salary
1	100	Steven	King	24,000
2	116	Shelli	Baida	2,900
3	117	Sigal	Tobias	2,800
4	123	Shanta	Vollman	6,500
5	128	Steven	Markle	2,200
6	138	Stephen	Stiles	3,200
7	150	Sean	Tucker	10,000

```
select
    employee_id,
    first_name,
    last_name,
    salary
from employees
where
        first_name like 'S%'
```

LIKE(3/3)

	employee_id	first_name	last_name	salary	department_id
1	105	David	Williams	4,800	60
2	106	Valli	Jackson	4,800	60
3	108	Nancy	Gruenberg	12,008	100
4	109	Daniel	Faviet	9,000	100
5	119	Karen	Colmenares	2,500	30
6	120	Matthew	Weiss	8,000	50
7	122	Payam	Kaufling	7,900	50

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
        first_name like '_a%'
```

BETWEEN

Operator BETWEEN w SQL służy do filtrowania wierszy na podstawie zakresu wartości dla określonej kolumny. Pozwala sprawdzić, czy wartość w danej kolumnie mieści się w podanym zamkniętym przedziale, czyli łącznie z wartościami granicznymi.

Jest to wygodniejszy i bardziej czytelny sposób na sprawdzanie zakresów niż używanie kombinacji operatorów porównania (>= i <=).

	123 employee_id	A-Z first_name	A-Z last_name	123 salary	123 department_id
1	104	Bruce	Miller	6,000	60
2	105	David	Williams	4,800	60
3	106	Valli	Jackson	4,800	60
4	111	Ismael	Sciarra	7,700	100
5	112	Jose Manuel	Urman	7,800	100
6	113	Luis	Popp	6,900	100
7	120	Matthew	Weiss	8,000	50

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
        salary between 4800 and 8000
```

AND, OR

	employee_id	first_name	last_name	salary	department_id
1	100	Steven	King	24,000	90
2	101	Neena	Yang	17,000	90
3	102	Lex	Garcia	17,000	90
4	103	Alexander	James	9,000	60
5	104	Bruce	Miller	6,000	60
6	108	Nancy	Gruenberg	12,008	100
7	109	Daniel	Faviet	9,000	100
8	110	John	Chen	8,200	100

```sql
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
    salary > 5000
    and department_id <> 30
    or department_id = 50
```

IN

Operator IN w SQL służy do sprawdzania, czy wartość w określonej kolumnie pasuje do którejkolwiek z wartości wymienionych na liście.

Jest to skrócony i bardziej czytelny sposób na łączenie wielu warunków OR z porównaniami równości (=).

Zamiast pisać wiele warunków typu kolumna = wartość1 OR kolumna = wartość2 OR kolumna = wartość3, możesz użyć operatora IN z listą tych wartości.

	123 employee_id	A-Z first_name	A-Z last_name	123 salary	123 department_id
1	114	Den	Li	11,000	30
2	115	Alexander	Khoo	3,100	30
3	116	Shelli	Baida	2,900	30
4	117	Sigal	Tobias	2,800	30
5	118	Guy	Himuro	2,600	30
6	119	Karen	Colmenares	2,500	30
7	200	Jennifer	Whalen	4,400	10
8	201	Michael	Martinez	13,000	20
9	202	Pat	Davis	6,000	20

```sql
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
        department_id IN (10,20,30)
```

NULL

Wartość NULL w SQL reprezentuje brakującą, nieznaną, nieokreśloną lub nieaplikowalną wartość danych w kolumnie tabeli.

Nie jest to zero, nie jest to pusty ciąg znaków ('\'), a nawet nie jest to spacja (' ,).

NULL to unikalny znacznik, który wskazuje na nieistnienie wartości.

Możesz sobie wyobrazić, że pusta komórka w arkuszu kalkulacyjnym niekoniecznie oznacza zero. Może oznaczać, że dane nie zostały jeszcze wprowadzone, są nieznane lub po prostu nie mają zastosowania w danym kontekście.

W SQL NULL pełni podobną rolę.

Kluczowe cechy i konsekwencje używania wartości NULL:

- Nie jest wartością: Kluczowe jest zrozumienie, że NULL nie jest wartością w tradycyjnym sensie. Dlatego nie można go porównywać za pomocą standardowych operatorów porównania (=, <>, >, <). Próba porównania z NULL zawsze da w wyniku UNKNOWN.
- Aby sprawdzić, czy wartość w kolumnie jest NULL, używa się specjalnych operatorów:
  - **IS NULL:** Sprawdza, czy wartość jest NULL.
  - **IS NOT NULL:** Sprawdza, czy wartość nie jest NULL.

IS NULL

	employee_id	first_name	last_name	salary	commission_pct
1	100	Steven	King	24,000	[NULL]
2	101	Neena	Yang	17,000	[NULL]
3	102	Lex	Garcia	17,000	[NULL]
4	103	Alexander	James	9,000	[NULL]
5	104	Bruce	Miller	6,000	[NULL]
6	105	David	Williams	4,800	[NULL]
7	106	Valli	Jackson	4,800	[NULL]
8	107	Diana	Nguyen	4,200	[NULL]

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    commission_pct
from employees
where
        commission_pct is null
```

NEGACJE

IS NOT NULL
NOT BETWEEN
NOT IN
NOT LIKE

HIERARCHIA OPERATORÓW

W warunkach **ZAWSZE** pierwszy sprawdzany jest AND potem OR.

Można zmienić tę kolejność poprzez zastosowanie ()

Obsługa NULL(1/3)

	123 employee_id	A-Z last_name	123 salary	123 commission_pct	123 result
1	100	King	24,000	[NULL]	[NULL]
2	101	Yang	17,000	[NULL]	[NULL]
3	102	Garcia	17,000	[NULL]	[NULL]
4	103	James	9,000	[NULL]	[NULL]
5	104	Miller	6,000	[NULL]	[NULL]
6	105	Williams	4,800	[NULL]	[NULL]
7	106	Jackson	4,800	[NULL]	[NULL]
8	107	Nguyen	4,200	[NULL]	[NULL]

```
select
    employee_id,
    last_name,
    salary,
    commission_pct,
    salary + commission_pct result
from employees
```

Obsługa NULL(2/3)

Funkcja COALESCE w PostgreSQL (i w wielu innych systemach baz danych SQL) służy do zwracania pierwszej nie-NULLowej wartości z listy podanych argumentów.

Przyjmuje ona zmienną liczbę argumentów i przetwarza je w kolejności, aż znajdzie wartość, która nie jest NULL.

Jak tylko taka wartość zostanie znaleziona, funkcja ją zwraca i przestaje przetwarzać pozostałe argumenty. Jeśli wszystkie podane argumenty są NULL, funkcja COALESCE również zwraca NULL.

	123 employee_id	A-Z first_name	A-Z last_name	123 salary	123 commission_pct	123 result
40	139	John	Seo	2,700	[NULL]	2,700
41	140	Joshua	Patel	2,500	[NULL]	2,500
42	141	Trenna	Rajs	3,500	[NULL]	3,500
43	142	Curtis	Davies	3,100	[NULL]	3,100
44	143	Randall	Matos	2,600	[NULL]	2,600
45	144	Peter	Vargas	2,500	[NULL]	2,500
46	145	John	Singh	14,000	0.4	14,000.4
47	146	Karen	Partners	13,500	0.3	13,500.3
48	147	Alberto	Errazuriz	12,000	0.3	12,000.3
49	148	Gerald	Cambrault	11,000	0.3	11,000.3

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    commission_pct,
    salary + COALESCE(commission_pct,0) result
from employees
```

Obsługa NULL(3/3)

```sql
create table coalesce_problem (col1 int, col2 int, col3 int);

insert into coalesce_problem values (null,null,1);
insert into coalesce_problem values (null,2,1);
insert into coalesce_problem values (3,2,1);

select * from coalesce_problem
```

	123 col1	123 col2	123 col3
1	[NULL]	[NULL]	1
2	[NULL]	2	1
3	3	2	1

```sql
select
    col1,
    col2,
    col3,
    coalesce(col1,col2,col3) result
from coalesce_problem
```

	123 col1	123 col2	123 col3	123 result
1	[NULL]	[NULL]	1	1
2	[NULL]	2	1	2
3	3	2	1	3

ORDER BY (1/4)

	employee_id	first_name	last_name	salary
1	121	Adam	Fripp	8,200
2	196	Alana	Walsh	3,100
3	147	Alberto	Errazuriz	12,000
4	115	Alexander	Khoo	3,100
5	103	Alexander	James	9,000
6	185	Alexis	Bull	4,100
7	158	Allan	McEwen	9,000
8	175	Alyssa	Hutton	8,800

```
select
    employee_id,
    first_name,
    last_name,
    salary
from employees
order by first_name
```

ORDER BY (2/4)

	employee_id	first_name	last_name	salary
1	100	Steven	King	24,000
2	101	Neena	Yang	17,000
3	102	Lex	Garcia	17,000
4	145	John	Singh	14,000
5	146	Karen	Partners	13,500
6	201	Michael	Martinez	13,000
7	205	Shelley	Higgins	12,008

```
select
    employee_id,
    first_name,
    last_name,
    salary
from employees
order by salary DESC
```

ORDER BY (3/4)

	last_name	department_id	salary
1	Whalen	10	4,400
2	Martinez	20	13,000
3	Davis	20	6,000
4	Li	30	11,000
5	Khoo	30	3,100
6	Baida	30	2,900

```
select
last_name,
department_id,
salary
from employees
order by department_id, salary DESC
```

ORDER BY (4/4)

	last_name	department_id	salary
1	Abel	80	11,000
2	Ande	80	6,400
3	Atkinson	50	2,800
4	Baida	30	2,900
5	Banda	80	6,200
6	Bates	80	7,300
7	Bell	50	4,000

```sql
select
    last_name,
    department_id,
    salary
from employees
order by 1
```

DISTINCT

Poniższy przykład wykorzystuje DISTINCT, aby zapobiec pobieraniu zduplikowanych wierszy.

	123 department_id
1	20
2	50
3	30
4	110
5	70
6	100
7	[NULL]
8	80

```
select
       distinct department_id
from employees
```

Kolejność wykonywania klauzul (1/3)

```
SQL Error [42703]: BŁĄD: kolumna "result" nie istnieje
  Position: 112

Error position: line: 8 pos: 111
```

```sql
select
    employee_id,
    first_name,
    last_name,
    department_id,
    salary + 100 result
from employees
where result > 10000
```

Kolejność wykonywania klauzul (2/3)

	employee_id	first_name	last_name	department_id	result
1	132	TJ	Olson	50	2,200
2	128	Steven	Markle	50	2,300
3	136	Hazel	Philtanker	50	2,300
4	135	Ki	Gee	50	2,500
5	127	James	Landry	50	2,500
6	140	Joshua	Patel	50	2,600
7	119	Karen	Colmenares	30	2,600

```sql
select
    employee_id,
    first_name,
    last_name,
    department_id,
    salary + 100 result
from employees
order by result
```

Kolejność wykonywania klauzul (3/3)

1 – **FROM**
2 – **WHERE**
3 – **GROUP BY**
4 – **HAVING**
5 – **ORDER BY**
6 – **SELECT**